

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning
FACULTY: Dr. Arul Raj A.M

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I **Ch. Venkata Lakshmi Hymavathi (192425397)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Ch. Hymavathi

Name of the student:

Ch. Venkata Lakshmi Hymavathi

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer:

Gradient Descent updates model parameters as:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

where:

- θ = model parameters
- η = learning rate
- $J(\theta)$ = loss function

Effect of High Initial Learning Rate

A **high initial learning rate** allows the algorithm to:

- Move quickly toward the optimum.
- Escape shallow local regions or plateaus.
- Reduce training time during early iterations.

As training progresses, **learning rate decay** gradually reduces the step size, enabling:

- Stable convergence.
- Fine adjustment near the minimum.
- Lower oscillations around the optimum.

When It Outperforms a Constant Learning Rate

This strategy performs better when:

- The loss function is smooth and convex.
- The initial point is far from the optimum.
- The learning rate decreases appropriately (step decay, exponential decay, cosine decay).

A constant learning rate:

- May converge slowly if too small.
- May oscillate around the optimum if too large.

Learning-rate decay combines **fast initial learning** with **accurate final convergence**.

Risks

- Too large an initial learning rate may:
 - Overshoot the minimum.
 - Cause divergence.
 - Produce unstable training.
- Too rapid decay may:
 - Stop learning prematurely.
 - Converge to a suboptimal solution.

Conclusion

A high initial learning rate followed by gradual decay provides faster convergence and better optimization than a constant learning rate when the decay schedule is chosen appropriately. However, improper tuning may lead to instability or poor convergence.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
 - The sample size increases
- What does this imply about the robustness of MLE?

Answer:

Suppose

$$X_1, X_2, \dots, X_n \sim N(\mu, \sigma^2)$$

where variance σ^2 is known.

Likelihood Function

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}$$

Taking logarithm,

$$\ell(\mu) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum (x_i - \mu)^2$$

Differentiate:

$$\frac{d\ell}{d\mu} = \frac{1}{\sigma^2} \sum (x_i - \mu)$$

Set equal to zero:

$$\sum (x_i - \mu) = 0$$

Hence,

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i$$

Thus, the MLE of the mean is the **sample mean**.

Effect of Outliers

The sample mean is highly sensitive to extreme values.

- One large outlier significantly shifts the estimate.
- Therefore, the MLE is **not robust** to outliers.

Effect of Increasing Sample Size

As sample size increases:

- Estimate becomes more accurate.
- Variance decreases.

$$Var(\hat{\mu}) = \frac{\sigma^2}{n}$$

Thus,

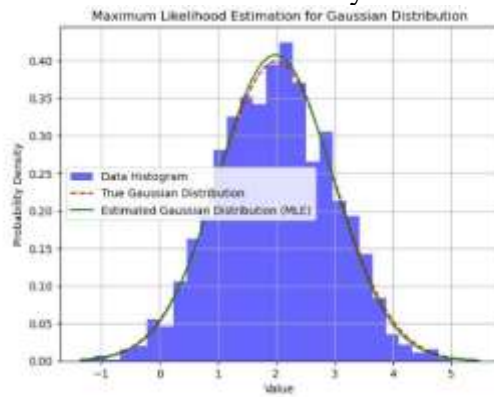
larger $n \Rightarrow$ smaller variance $\Rightarrow$ better estimation.

Implication

MLE is:

- Efficient for clean Gaussian data.
- Consistent as sample size grows.

- Sensitive to outliers because every observation contributes equally.



3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer:

Challenges

Dataset:

- Features = 10,000
- Samples = 500

This creates a **high-dimensional, low-sample-size** problem.

Challenges include:

- Overfitting
- High computational cost
- Multicollinearity
- Curse of dimensionality
- Poor generalization

Proposed Algorithm Pipeline

Step 1: Data Preprocessing

- Remove missing values.
- Normalize features.

Reason:

- Ensures consistent feature scales.

Step 2: Feature Selection

Methods:

- Chi-square

- Mutual Information
- LASSO

Reason:

- Remove irrelevant features.
- Reduce dimensionality.

Step 3: Dimensionality Reduction

Use PCA.

Reason:

- Compress correlated features.
- Preserve most variance.
- Reduce computation.

Step 4: Regularization

Use:

- L1 (LASSO)
- L2 (Ridge)

Reason:

- Penalizes complex models.
- Reduces overfitting.

Step 5: Train Classifier

Examples:

- Support Vector Machine (SVM)
- Logistic Regression
- Random Forest

Step 6: Cross Validation

Use k-fold cross-validation.

Reason:

- Better estimation of model performance.
- Avoids dependence on one train-test split.

How Overfitting is Reduced

- Feature selection removes noise.
- PCA reduces dimensionality.
- Regularization limits model complexity.
- Cross-validation improves generalization.

Conclusion

Combining feature selection, dimensionality reduction, regularization, and cross-validation produces a simpler model that generalizes better despite having far more features than training samples.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

A Multilayer Perceptron (MLP) consists of:

- Input layer
- Hidden layer
- Output layer

Each hidden neuron computes:

$$y = \sigma(Wx + b)$$

where

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

is the sigmoid activation function.

Why More Hidden Neurons Increase Representational Power

Each hidden neuron learns different nonlinear patterns.

More neurons allow the network to:

- Learn more complex decision boundaries.
- Approximate complicated nonlinear functions.
- Capture richer feature interactions.

According to the Universal Approximation Theorem, an MLP with enough hidden neurons can approximate any continuous function.

Why More Neurons Do Not Always Improve Performance

Increasing neurons also increases:

- Number of parameters
- Training time
- Model complexity

If the model becomes too complex:

- It memorizes training data.
- Noise is learned instead of meaningful patterns.
- Test accuracy decreases.

This is called **overfitting**.

Generalization

Generalization means performing well on unseen data.

Good generalization requires balancing:

- Model complexity
- Training accuracy

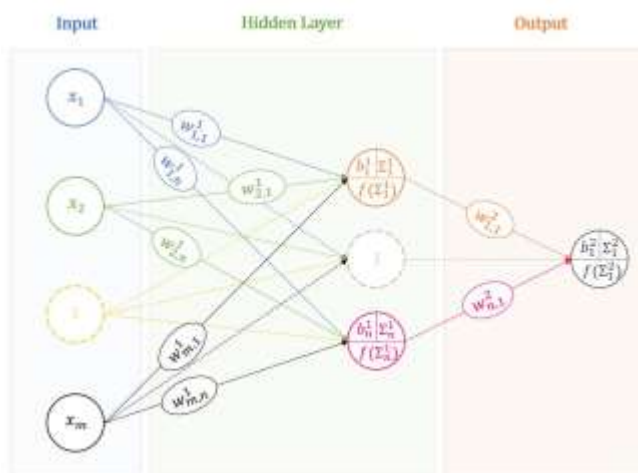
Techniques such as:

- Dropout
- Early stopping
- Regularization
- Cross-validation

help improve generalization.

Conclusion

Adding hidden neurons increases learning capacity but excessive neurons may overfit the training data. The best performance is achieved by selecting an appropriate network size that balances representational power and generalization



5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

The curse of dimensionality occurs when the number of features becomes very large.

Impact on Gradient Updates

In high dimensions:

- Gradients become noisy.
- Some parameters receive very small gradients.
- Others receive very large gradients.

This makes optimization unstable.

Impact on SGD Convergence

Stochastic Gradient Descent may:

- Require many iterations.
- Converge slowly.
- Oscillate before reaching the optimum.

Sparse data in high dimensions also reduces the effectiveness of each gradient update.

Impact on Generalization

High-dimensional models often:

- Memorize training data.
- Learn noise.
- Produce high variance.

As a result:

- Training accuracy is high.
- Test accuracy is low.

Techniques to Overcome These Issues

1. Dimensionality Reduction (PCA)

- Reduces feature space.
- Removes redundant information.
- Speeds up training.
- Improves convergence.

2. Regularization (L1/L2)

- Penalizes large weights.
- Prevents overfitting.
- Improves generalization.

Additional Techniques

- Feature selection to remove irrelevant variables.
- Batch Normalization to stabilize gradients.
- Adaptive optimizers (Adam, RMSProp) for faster convergence.

- Early stopping to avoid overfitting.

Conclusion

The curse of dimensionality slows optimization, affects gradient updates, and reduces model generalization. Applying dimensionality reduction, regularization, and appropriate optimization techniques improves convergence speed and predictive performance.